开题报告二：面向海外AI接口的HTTPS反向代理系统

一、项目题目

面向海外AI接口的HTTPS反向代理系统
——基于套接字编程的HTTP/HTTPS中转代理实现

二、项目背景与研究意义

2.1 项目背景

海外主流AI服务（如大模型API、图像生成服务）存在区域访问限制，且跨境网络链路延迟高、稳定性差。现有中转方案多依赖Nginx、Cloudflare等现成工具，虽然部署便捷，但无法深入理解代理服务的底层网络原理，也不支持定制化的请求处理、流量控制与状态管理。

本项目基于网络课程核心知识点，**从零自主编程实现HTTPS反向代理**，不依赖第三方代理工具，从TCP层透明转发到HTTP协议解析，再到TLS加密传输，逐层实现完整的AI接口中转能力，既是对套接字编程、HTTP协议、TLS传输等知识点的综合实践，也具备实际使用价值。

2.2 研究意义

1. **课程实践意义**：覆盖TCP套接字编程、HTTP协议解析、TLS加密传输、多并发连接管理等核心知识点，深入理解反向代理的底层工作原理。
2. **技术价值**：自主可控的代理架构，支持请求头修改、路径重写、异常处理等定制化能力，比现成工具更便于二次开发。
3. **原理验证价值**：完整实现从TCP字节流到HTTP应用层的代理转发，直观验证传输层与应用层的协议封装与解析过程。

三、项目核心内容

项目以**代理转发逻辑为核心**，网络编程实现占比80%，AI业务场景占比20%。

3.1 TCP透明转发层（基础骨架）

1. **服务端监听**：实现标准TCP服务端流程，监听本地端口，等待客户端连接。
2. **目标端连接**：每接入一个客户端，同步创建与海外AI服务器的TCP连接。
3. **双向数据转发**：采用双线程模型，分别负责「客户端→目标服务器」「目标服务器→客户端」两个方向的字节流透明搬运。
4. **连接生命周期管理**：正常断开、异常掉线时同步关闭两端套接字，回收资源。

3.2 HTTP协议解析与代理层

1. **HTTP请求解析**：解析请求行、请求头、请求体，识别请求方法、URL、Host字段、Content-Length等关键信息。
2. **请求重写**：自动修改Host字段为目标AI服务域名，支持路径前缀重写，适配不同AI接口的路径规则。

3. **响应透传**：解析HTTP响应状态码、响应头，完整转发响应体，支持Transfer-Encoding分块响应。
4. **请求-响应匹配**：支持HTTP长连接复用，正确匹配多并发请求与响应，不串包不乱序。

3.3 TLS加密传输层

1. **服务端TLS可选**：支持客户端与代理之间使用HTTPS连接，完成TLS握手与加密解密。
2. **出站TLS加密**：代理与海外AI服务器之间建立标准TLS连接，兼容主流AI服务的HTTPS接口。
3. **证书校验**：支持目标服务器证书合法性校验，防止中间人攻击。

3.4 连接管理与异常处理

1. **连接超时控制**：目标服务器连接超时、数据传输超时自动断开，返回明确错误提示。
2. **错误透传**：目标服务不可达、接口报错等场景，向客户端返回标准HTTP错误码与错误信息。
3. **基础访问控制**：支持IP白名单配置，拒绝未授权IP接入。

四、对应难度等级与判定依据

对应难度：高级（难度系数1.2）

判定依据

1. **协议深度匹配**：实现HTTP应用层协议解析与重写，对应难度表中「REST API服务、应用层协议处理」高级技术点。
2. **加密能力达标**：集成TLS/SSL实现加密传输，对应难度表中「TLS/SSL安全通道」高级核心特征。
3. **并发与工程性**：多连接并发管理、完整的异常处理与状态管理，覆盖课程高级阶段的工程实践要求。
4. **功能完整性**：从TCP透明转发到HTTP代理再到TLS加密，形成完整的代理能力，功能可直接对接真实AI服务接口。

五、基础实现方案与技术路线

5.1 技术栈

- 编程语言：C语言
- 网络接口：Linux POSIX Socket API
- 加密库：OpenSSL
- 验证工具：Wireshark抓包、curl接口测试

5.2 实施步骤

1. **阶段一：TCP透明转发实现（1周）**
   完成TCP服务端监听、目标连接、双向数据转发核心逻辑，实现端口级透明中转，验证基础AI接口连通性。
2. **阶段二：HTTP代理实现（1.5周）**
   实现HTTP请求与响应解析、Host重写、路径转发，支持完整的HTTP/1.1协议代理。
3. **阶段三：TLS加密集成（1.5周）**
   集成OpenSSL，实现出站HTTPS连接，可选支持入站HTTPS，完成加密链路的完整转发。

4. **阶段四：测试优化与文档整理（1周）**
   完成真实AI接口对接测试、并发性能测试、异常场景测试，整理项目代码与说明文档。

六、可拓展研究方向

1. **多节点负载均衡**：配置多个海外中转节点，根据延迟、负载自动选择最优节点，支持故障自动切换，升级至挑战级难度。
2. **请求缓存加速**：对高频请求（如模型列表、常用问答）做结果缓存，减少重复转发，降低访问延迟。
3. **epoll高并发重构**：将多线程模型替换为epoll事件驱动架构，提升单服务器并发承载量。
4. **请求限流与管控**：新增IP级、用户级请求频率限制，保障服务稳定性，防止接口滥用。
5. **内容审计与过滤**：对请求与响应内容做基础审计，拦截违规请求。
6. **链路优化**：针对跨境长延迟链路，调整TCP缓冲区、拥塞控制参数，优化传输性能。

七、同类参考项目（GitHub / CSDN / Gitee）

> 下列项目与本选题「TCP 转发 + HTTP 解析重写 + OpenSSL/TLS 加密 + 多连接并发」的技术路线高度重合，可作为架构与实现层面的直接参照。

7.1 高度类似的开源项目（按相关度排序）

1. **ngrav007/https-proxy（C 语言，最贴近本选题）**
   用 C 语言实现的完整 HTTP/HTTPS 代理，含 HTTP 服务端与客户端两套实现，核心即「Socket 通信 + OpenSSL 完成 SSL/TLS 加解密」，与本题 3.1 TCP 转发层、3.3 TLS 加密层几乎一一对应。
   https://github.com/ngrav007/https-proxy
2. **once-ai/HTTP-ProxyServer（C 语言，访问控制形态最像）**
   基于 Socket 的 HTTP 代理服务器，Linux 环境、C 语言开发，实现了**客户端 IP 校验 + 特定目标服务器排除 + HTTP 请求解析转发**，正对应本题 3.4 节「基础访问控制（IP 白名单）」与 3.2 节请求转发。
   https://github.com/once-ai/HTTP-ProxyServer
3. **tinyproxy/tinyproxy（工业级轻量实现，原稿提到的 TinyProxy 本体）**
   POSIX 系统上的轻量级 HTTP/HTTPS 代理守护进程，纯 C 编写、代码量可控，是理解「真实可用的代理服务如何组织连接管理与配置」的最佳范本，也便于对照自研实现的差距。
   https://github.com/tinyproxy/tinyproxy
4. **Yello666/RPS（C 语言，反向代理形态）**
   纯 C 实现的轻量级**反向代理**服务器，支持 HTTP/1.1，与本题的「反向代理」定位一致，可参考其请求解析与后端转发逻辑。
   https://github.com/Yello666/RPS
5. **wuzhiyu89/Multi-Threaded-HTTP-Proxy-Server（C 语言，并发与缓存）**
   多线程 HTTP 代理服务器，含线程池、哈希表缓存与 LRU 淘汰机制，可直接对照本题拓展方向 2「请求缓存加速」与拓展方向 3「并发重构」。
   https://github.com/wuzhiyu89/Multi-Threaded-HTTP-Proxy-Server
6. **liuyuyan6100/mproxy（Gitee，微型最小参照）**
   核心代码不足 500 行、不依赖任何第三方库的微型 HTTP 代理，适合作为「最小可运行参照」先跑通整体流程，再逐步叠加解析、TLS 与访问控制。
   https://gitee.com/liuyuyan6100/mproxy